```
21.3.1; however, version 22.0.4 is
available.
You should consider upgrading via the
'C:\Users\kamil\Envs\venv1\Scripts\
python.exe -m pip install --upgrade
pip' command.
```

Let us now create a table in our database. These 'tables' are referred to, as per the terminology of a Django project, as 'models', and are defined within the 'models.py' file in the app. Within 'models.py' of *firstapp*, let's write the following:

```
from django.db import models

# Create your models here.

class Studentinfo(models.Model):
    First_name=models.CharField(max_len
gth=50,blank=False,null=False)
    Last_name=models.CharField(max_leng
th=50,blank=False,null=False)
    Student_ID=models.IntegerField(blan
k=False,null=False)
    Email_ID=models.EmailField(blank=Fa
lse,null=False)
```

Our idea is to create a simple table/model taking in the information of a student. The fields 'First_name' and 'Last_name' are character fields, which allow a maximum length of 50 characters. The 'Required=True' parameter implies that you cannot discard this field (this will be covered in a case scenario later). The 'null=False' means that these fields will not accept null values. There are many different types of fields and parameters for Django models, which you can find in the official documentation of Django.

We now need to 'send' our defined model/table to our database. For this, let's make a few changes to the 'DATABASES' section within the *settings.py* file of 'osfyproject1' in VS Code. The pre-existing section looks as follows:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.
sqlite3',
        'NAME': BASE_DIR / 'db.
sqlite3',
    }
}
```

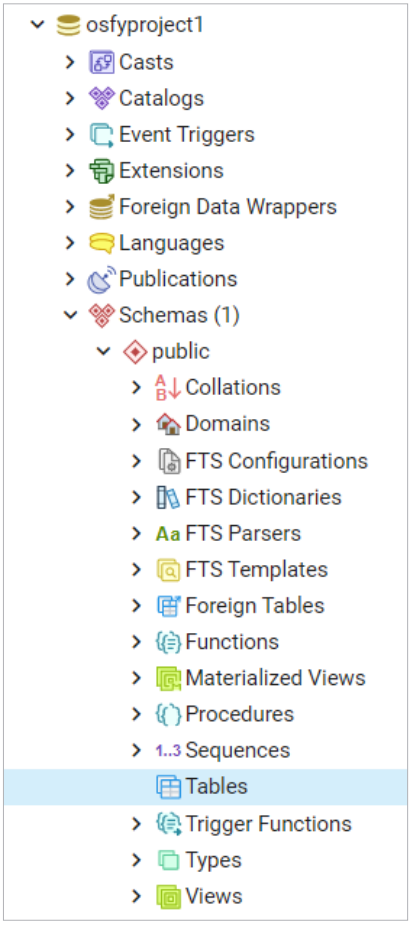

Figure 6: Newly created database in pgAdmin

By default, it considers sqlite3 as its database platform, but in our case, we are using PostgreSQL. The modified section should appear like this:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.
postgresql',
        'NAME': 'osfyproject1',
        'USER':'postgres',
        'PASSWORD':'password',
        'HOST':'localhost',
    }
}
```

The 'PASSWORD' will be the password you set when you installed PostgreSQL on your system (mine was simply 'password' itself).

Another section within 'settings.py' which requires an addition is the 'INSTALLED_APPS' section.

```
INSTALLED_APPS = [
    'firstapp',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

Kindly add 'firstapp' in the above list.

Now, we must send our table to the database, which in this terminology is 'migrating' our models. Migrating our defined model to our PostgreSQL database involves two simple steps:

```
(venv1) C:\Users\kamil\osfydir\
osfyproject1>python manage.py
makemigrations
Migrations for 'firstapp':
  firstapp\migrations\0001_initial.py
    - Create model Studentinfo
```

The 'makemigrations' step automatically creates a file called '0001_initial.py' in the *migrations* folder within our 'firstapp'. This file has code written in Python, which inculcates the definition of our model in a way that PostgreSQL will understand. Let's take a peek at how this looks:

```
from django.db import migrations,
models

class Migration(migrations.Migration):
    initial = True
    dependencies = [
```